

ازاي كود ال C# بيتحول ل SQL Query

1. ال LINQ أصلاً بتتعامل مع إيه؟

الـ LINQ بتوفر طريقة type-safe ومنظمة لكتابة queries على local collections وعلى remote data sources.

الـ basic units هي:

Sequence



Elements

وأي sequence بتطبق `IEnumerable<T>` تقدر تستخدم عليها LINQ operators.

الـ query operator بياخد sequence ويطلع sequence جديدة متحوّرة منها.

2. فيه عالمين مهمين في LINQ

Local Queries

المصدر:

`IEnumerable<T>`

والـ operators من:

`Enumerable`

والـ lambda بتتحول لـ delegates زي:

`Func<T, ...>`

والـ execution بيحصل locally داخل الـ CLR.

IEnumerable<T>



Enumerable.*



Func<T, ...>



CLR



Objects

Interpreted Queries

المصدر:

IQueryable<T>

والoperators من:

Queryable

والlambda بتمثل ك:

Expression<Func<T, ...>>

بحيث الquery provider يقدر يفحص الquery ويترجمها للغة ثانية، زي SQL.

IQueryable<T>



Queryable.*



Expression Trees



Query Provider



SQL

3. **ليه IQueryable** موجودة؟

client. لكن ساعتها التنفيذ ممكن يحصل على الـ **IQueryable** ممكن تتعامل مع **operators** **Enumerable**.

Queryable remote/provider-based execution. موجوده علشان تسمح بالـ

يعني :

Enumerable
↓
Local execution

مقابل :

Queryable
↓
Provider execution

4. مثال مع EF Core

```
var query =  
    dbContext.Customers  
        .Where(c => c.Name.Contains("a"))  
        .Select(c => c.Name);
```

المصدر هنا **<DbSet<Customer>** والـ provider هو EF Core.

الـ LINQ query ممكن تتحول لـ SQL، والـ SQL يتنفذ على database، والنتيجة ترجع كـ objects.

LINQ
↓
EF Core
↓
SQL
↓
Database
↓
Results

5. إيه اللي بيحصل وقت Compilation؟

الـ book بيوصف العملية كده :

Query Syntax
↓
Fluent Syntax
↓
Resolve Extension Methods
↓
IQueryable?
↓
Queryable Operators
↓
Expression Trees
↓
Complete Expression Tree

لو ال source هو `IQueryable<T>` ، ال compiler يختار operators `Queryable` بدل `Enumerable` ، وال lambdas تتحول إلى Expression Trees.

6. ال Expression Trees

ال Expression Tree يتمثل ال expression ك object model.

مثلاً:

```
c => c.Age >= 18
```

ممکن تتمثل ك structure يقدر ال provider يفحصها.

ال Expression Trees موجودة في:

```
System.Linq.Expressions
```

Mental model:

C# Expression
↓
Expression Tree
↓
Object Model
↓
Provider Inspection



Translation

7. Func VS Expression<Func<>>

في Local LINQ :

```
Func<Customer, bool>
```

دي executable behavior .

أما في interpreted LINQ :

```
Expression<Func<Customer, bool>>
```

دي representation لل-expression .

علشان كده :

```
var employees =  
    context.Employees  
        .Where(e => e.Salary > 10000);
```

ممكن ال-provider يتعامل معها .

لكن :

```
bool IsHighSalary(Employee e)  
{  
    return e.Salary > 10000;  
}  
  
var employees =  
    context.Employees  
        .Where(IsHighSalary);
```

ال-provider شايف method call، ومش ببساطة بيقرا implementation بتاعة method ويحول body بتاعتها لـ SQL .

الخلاصة :

```
IEnumerable<T>
  ↓
Func
  ↓
Execute code

IQueryable<T>
  ↓
Expression<Func<...>>
  ↓
Describe code
  ↓
Provider translates it
```

8. Query Provider

بعد ما الـ query تتمثل كـ Expression Tree، الـ provider يعالج الـ query.

مع EF Core :

```
Expression Tree
  ↓
EF Core
  ↓
Translation
  ↓
SQL
  ↓
Database
  ↓
Results
```

لكن الـ provider مش بيقدر يترجم أي C# expression.

الـ providers بتفهم expressions مدعومة فقط، وبعض LINQ queries الصحيحة ممكن مايقاش لها SQL translation، وساعتها ممكن تحصل runtime error.

9. Deferred Execution

الـ Deferred Execution يستخدم interpreted queries.

مثلاً:

```
var query =  
    dbContext.Customers  
        .Where(c => c.Name.Contains("a"));
```

إنشاء الـ query مش معناه إن database query اتنفذت.

التنفيذ يحصل لما الـ query يتم enumerate:

```
var result = query.ToList();
```

أو:

```
foreach (var customer in query)  
{  
}
```

Mental model:

```
Build Query  
  ↓  
Deferred  
  ↓  
Enumeration  
  ↓  
Provider Processing  
  ↓  
SQL  
  ↓  
Database
```

10. Multiple Enumeration

لو عندك:

```
var query =  
    dbContext.Customers
```

```
.Where(c => c.IsActive);
```

وبعدين :

```
query.ToList();  
query.ToList();
```

أنت عملت enumeration مرتين.

Conceptually:

```
Enumeration #1  
    ↓  
SQL Execution #1  
  
Enumeration #2  
    ↓  
SQL Execution #2
```

وده مهم جدًا مع database.

11. Combining Interpreted and Local Queries

يمكن query واحدة تحتوي على:

```
IQueryable
```

وبعدها :

```
IEnumerable
```

الشكل المعتاد :

```
Interpreted operators  
    ↓  
Provider / Database  
    ↓  
Boundary  
    ↓  
Local operators
```




Client

طول ما المصدر `<IQueryable<T>` ، الـ Queryable operators تفضل مستخدمة.

لما يحصل switch لـ `<IEnumerable<T>` ، الـ operators اللي بعدها تبقى `Enumerable`.

12. المهمة Boundary الـ – `AsEnumerable()`

مثال :

```
var query =
    dbContext.Customers
        .Where(c => c.Id > 10)
        .AsEnumerable()
        .Where(c => CustomMethod(c));
```

قبل `AsEnumerable()` :

Queryable.Where



Provider

بعدها :

Enumerable.Where



Local Execution

يعني `AsEnumerable()` بتغير operator binding من `Queryable` إلى `Enumerable`.

13. Query لا تنفذ الـ `AsEnumerable()`

دي نقطة أساسية.

`.AsEnumerable()`

لا تعني :

Execute now

ولا:

Create a List

هي بتعيد نفس sequence كـ `<IEnumerable<T>` علشان تغير overload resolution للـ operators اللي بعدها.

```
IQueryable<T>
    ↓
AsEnumerable()
    ↓
IEnumerable<T>
```

مش:

```
IQueryable<T>
    ↓
Execute
    ↓
List
```

14. إمتى نستخدم `AsEnumerable()` ؟

لما provider يكون مش قادر يترجم جزء من الـ query.

مثال باستخدام `Regex`:

```
Regex regex = new Regex(...);
```

الجزء ده ممكن ماييقاش له SQL translation:

```
dbContext.Articles
    .Where(a => regex.Matches(a.Text).Count < 100);
```

فنقدر نفصل الـ query:

```
dbContext.Articles
    .Where(a => a.Topic == "influenza")
    .AsEnumerable()
    .Where(a => regex.Matches(a.Text).Count < 100);
```

فتبقى :

Database Side

↓

```
.Where(a => a.Topic == "influenza")
```

↓

```
AsEnumerable()
```

↓

Client Side

↓

Regex

الفكرة: خُلي الـ database تعمل الجزء اللي الـ provider فاهمه، وبعد كده نَفِّذ الجزء المحلي في C#.

15. Performance

خلي `AsEnumerable()` متأخر قدر الإمكان.

الأفضل :

Database

↓

Translatable operations

↓

```
AsEnumerable()
```

↓

Unsupported/local operation

↓

Client

مش :

```
AsEnumerable()
```

↓

Large dataset



Client-side filtering

لأن نقل data زيادة للclient ممكن يقلل performance بشكل كبير.

16. AsEnumerable() vs ToList()

AsEnumerable()

.AsEnumerable()

- لا تعمل immediate execution.
- لا تعمل materialization.
- لا تنشئ collection جديدة.
- تغيّر الـ operator binding من Queryable إلى Enumerable.

ToList()

.ToList()

- تعمل execution.
- تحمل النتائج في memory.
- ترجع `<List<T>`.

Mental model:

AsEnumerable()

IQueryable



IEnumerable view



Continue locally

مقابل :

ToList()

IQueryable

↓
Execute
↓
Materialize
↓
List<T>

17. ليه Custom Query Operators صعبة؟

عمل custom LINQ operator للـ local collections سهل نسبيًا.

لكن لو عايز custom interpreted operator، الـ provider لازم يفهمه ويعرف يترجمه.

زي Standard operators:

Where
Select
OrderBy
Join
GroupBy

معروفة للـ provider.

لكن:

MyWhere()

مش معناها تلقائيًا إن EF Core هيعرف يحولها لـ SQL.

18. الرحلة الكاملة

C# LINQ
↓
Source Type
↓

IEnumerable<T>

Enumerable operators

Func delegates
Local execution

OR

IQueryable<T>
Queryable operators
Expression Trees
Query Provider
Remote / interpreted query

↓
Translation
↓
SQL
↓
Database
↓
Results

ولو استخدمت :

.AsEnumerable()

بيبقى عندك :boundary

IQueryable
↓
Provider / Database
↓
AsEnumerable()
↓
IEnumerable
↓
CLR / Client

لما تشوف:

```
var query =  
    dbContext.Customers  
        .Where(...)  
        .Select(...)  
        .OrderBy(...);
```

ما تقولش:

"أنا جيت الـ Customers."

قول:

"أنا بنيت interpreted query."

ولما تشوف:

```
query.ToList();
```

قول:

"أنا طلبت enumeration، وبالتالي الـ provider عنده فرصة ينفذ الـ query."

ولما تشوف:

```
AsEnumerable()
```

قول:

"أنا بغير الـ boundary؛ الـ operators اللي بعدها هتتربط بـ Enumerable بدل Queryable."

ولما تشوف:

```
Expression<Func<T, bool>>
```

قول:

"الـ lambda هنا representation يقدر الـ provider يفحصها."

C# → SQL

لكن بشكل conceptual :

```
LINQ
  ↓
IQueryable
  ↓
Queryable Operators
  ↓
Expression Tree
  ↓
Query Provider
  ↓
Translation
  ↓
SQL
  ↓
Database
  ↓
Results
```

ولو دخلت `AsEnumerable()` :

```
Provider / Database
  ↓
AsEnumerable()
  ↓
IEnumerable
  ↓
Local LINQ
```

أهم 4 أفكار :

```
IEnumerable
→ Local LINQ

IQueryable
→ Interpreted/provider-based LINQ
```


Expression<Func<...>>

→ Representation لـ expression يقدر provider بفحصها

AsEnumerable()

→ Boundary إلى Enumerable Queryable التي بعدها من operators تتغير الـ

والتالي Deferred Execution معناها إن interpreted query بتستنى enumeration قبل التنفيذ، وبالتالي repeated enumeration ممكن يؤدي لتنفيذ الـ query أكثر من مرة.

GB Part II – English Deep Dive

From LINQ to SQL – The Journey of a Query

1. LINQ

LINQ provides language and runtime features for writing structured, type-safe queries over local object collections and remote data sources.

The fundamental concepts are:

Sequence



Elements

A sequence can be represented by `IEnumerable<T>`, while query operators transform sequences into new sequences.

2. Two LINQ Architectures

Local Queries

IEnumerable<T>



Enumerable.*



Func<T, ...>



CLR

↓
Objects

Interpreted Queries

IQueryable<T>
↓
Queryable.*
↓
Expression Trees
↓
Query Provider
↓
SQL

The distinction is not merely an interface difference. It determines whether LINQ is executed locally or represented for provider-based execution.

3. Why **Queryable** Exists

Enumerable operators can operate against an **IQueryable** source, but that means the operation is performed locally.

Queryable exists to support remote/provider-based execution.

Enumerable
↓
Local execution

versus:

Queryable
↓
Provider execution

4. EF Core

Consider:

```
var query =  
    dbContext.Customers  
        .Where(c => c.Name.Contains("a"))  
        .Select(c => c.Name);
```

The source is an EF Core `DbSet<Customer>` .

The provider can process the LINQ query, translate it into SQL, execute that SQL against the database, and return the results as objects.

```
LINQ  
↓  
EF Core  
↓  
SQL  
↓  
Database  
↓  
Results
```

5. Compilation

Conceptually:

```
Query Syntax  
↓  
Fluent Syntax  
↓  
Extension Method Resolution  
↓  
IQueryable?  
↓  
Queryable Operators  
↓  
Expression Trees  
↓  
Complete Expression Tree
```

If the source is `IQueryable<T>` , the compiler selects `Queryable` operators instead of `Enumerable` , and the lambdas are represented as Expression

Trees.

6. Expression Trees

An Expression Tree represents an expression as an object model.

For example:

```
c => c.Age >= 18
```

can be represented as a structure that a provider can inspect.

Expression Trees are part of:

```
System.Linq.Expressions
```

Conceptually:

```
C# Expression
  ↓
Expression Tree
  ↓
Object Model
  ↓
Provider Inspection
  ↓
Translation
```

7. Func VS Expression<Func<>>

For local LINQ:

```
Func<Customer, bool>
```

represents executable behavior.

For interpreted LINQ:

```
Expression<Func<Customer, bool>>
```

represents the expression structure.

Therefore:

```
var employees =  
    context.Employees  
        .Where(e => e.Salary > 10000);
```

can be understood by a provider.

Whereas:

```
bool IsHighSalary(Employee e)  
{  
    return e.Salary > 10000;  
}  
  
var employees =  
    context.Employees  
        .Where(IsHighSalary);
```

presents the provider with a method call rather than the method body as an expression tree.

The mental model is:

```
IEnumerable<T>  
    ↓  
Func  
    ↓  
Execute code  
  
IQueryable<T>  
    ↓  
Expression<Func<...>>  
    ↓  
Describe code  
    ↓  
Provider translates it
```

8. Query Providers

Once the query is represented as an expression tree, the provider processes it.

With EF Core:

Expression Tree



EF Core



Translation



SQL



Database



Results

Providers have translation limits. Not every valid C# expression has a supported SQL translation.

9. Deferred Execution

Interpreted queries use deferred execution.

Creating:

```
var query =  
    dbContext.Customers  
        .Where(c => c.Name.Contains("a"));
```

does not mean that the database query has already executed.

Execution occurs when the query is enumerated:

```
var result = query.ToList();
```

or:

```
foreach (var customer in query)  
{  
}
```

Conceptually:

```
Build Query
  ↓
Deferred
  ↓
Enumeration
  ↓
Provider Processing
  ↓
SQL
  ↓
Database
```

10. Multiple Enumeration

Consider:

```
var query =
    dbContext.Customers
        .Where(c => c.IsActive);
```

Then:

```
query.ToList();
query.ToList();
```

The query is enumerated twice:

```
Enumeration #1
  ↓
SQL Execution #1

Enumeration #2
  ↓
SQL Execution #2
```

This is particularly important for remote data sources.

11. Combining Interpreted and Local Queries

A single query can contain both interpreted and local operations.

Interpreted operators



Provider / Database



Boundary



Local operators



Client

As long as the source is `IQueryable<T>`, `Queryable` operators can be selected.

After switching to `IEnumerable<T>`, subsequent operators bind to `Enumerable`.

12. AsEnumerable()

Consider:

```
var query =  
    dbContext.Customers  
        .Where(c => c.Id > 10)  
        .AsEnumerable()  
        .Where(c => CustomMethod(c));
```

Before `AsEnumerable()`:

Queryable.Where



Provider

After it:

Enumerable.Where



Local Execution

`AsEnumerable()` changes the binding of subsequent operators from `Queryable` to `Enumerable`.

13. `AsEnumerable()` Does Not Execute the Query

This is critical.

```
.AsEnumerable()
```

does not mean:

```
Execute now
```

and does not mean:

```
Create a List
```

It returns the sequence as `IEnumerable<T>` so that subsequent operators bind to `Enumerable`.

```
IQueryable<T>  
  ↓  
AsEnumerable()  
  ↓  
IEnumerable<T>
```

not:

```
IQueryable<T>  
  ↓  
Execute  
  ↓  
List
```

14. When `AsEnumerable()` Is Useful

Suppose a provider cannot translate an operation such as a particular `Regex` operation.

You can separate the query:

```
dbContext.Articles
    .Where(a => a.Topic == "influenza")
    .AsEnumerable()
    .Where(a => regex.Matches(a.Text).Count < 100);
```

Conceptually:

```
Database Side
  ↓
Translatable Where
  ↓
AsEnumerable()
  ↓
Client Side
  ↓
Regex
```

This allows the provider to execute what it understands before local processing begins.

15. Performance

A key rule is:

Call `**AsEnumerable()` as late as possible.

Prefer:

```
Database
  ↓
Translatable operations
  ↓
AsEnumerable()
  ↓
Unsupported/local operation
  ↓
Client
```

rather than:

```
AsEnumerable()  
  ↓  
Large dataset  
  ↓  
Client-side filtering
```

Moving unnecessary data to the client can significantly hurt performance.

16. `AsEnumerable()` vs `ToList()`

`AsEnumerable()`

```
.AsEnumerable()
```

- Does not immediately execute the query.
- Does not materialize a collection.
- Changes subsequent operator binding from `Queryable` to `Enumerable`.

`ToList()`

```
.ToList()
```

- Executes the query.
- Loads results into memory.
- Returns `List<T>`.

Mental model:

```
AsEnumerable()  
  
IQueryable  
  ↓  
IEnumerable view  
  ↓  
Continue locally
```

versus:

```

ToList()
  ↓
IQueryable
  ↓
Execute
  ↓
Materialize
  ↓
List<T>

```



17. Custom Query Operators

Final Mental Model



`IEnumerable<T>`

Enumerable operators

Func delegates

Local execution

OR

`IQueryable<T>`

Queryable operators

Expression Trees

Query Provider

Remote / interpreted query



Translation



SQL



Database



Results

And if the query crosses:

`.AsEnumerable()`

the boundary becomes:

`IQueryable`



Provider / Database



`AsEnumerable()`



`IEnumerable`



CLR / Client

Final Takeaway

The journey is not simply:

C# → SQL

A stronger conceptual model is:

```

LINQ
  ↓
IQueryable
  ↓
Queryable Operators
  ↓
Expression Tree
  ↓
Query Provider
  ↓
Translation
  ↓
SQL
  ↓
Database
  ↓
Results

```

The four ideas to remember:

IEnumerable
→ Local LINQ

IQueryable
→ Interpreted/provider-based LINQ

Expression<Func<...>>
→ Expression representation that a provider can inspect

AsEnumerable()
→ Boundary that changes subsequent operators from Queryable to Enumerable

Deferred execution means that an interpreted query waits for enumeration before execution, so repeated enumeration can lead to repeated query

execution.

The Curious Backend Developer

The goal is not to memorize LINQ.

The goal is to understand what the LINQ query actually means.